7

Direct Preference Optimization

From RLHF to DPO

Chapter 6 revealed both the power and the pain of classical RLHF. The power: a reward model trained on human comparisons can guide a policy to produce responses that are better than anything in its original training data. The pain: you need four models in GPU memory simultaneously, PPO has a dozen sensitive hyperparameters, and the reward model itself can be exploited through reward hacking.

In 2023, Rafael Rafailov and colleagues at Stanford asked a deceptively simple question: *what if we could skip the reward model entirely?* Their paper, "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model," showed that the answer was yes. The mathematical insight is elegant: the reward function that the RLHF pipeline learns is already encoded, implicitly, inside the policy itself. You do not need to extract it into a separate model. You can train the policy directly on preference pairs, with no reward model, no value function, no PPO loop.

The result is DPO: a method that achieves comparable performance to full RLHF while cutting memory requirements in half and replacing a complex RL training loop with

something that looks almost like supervised learning. This chapter derives DPO from the ground up, walks through the math with concrete numbers, and explains when DPO is the right choice and when it is not.

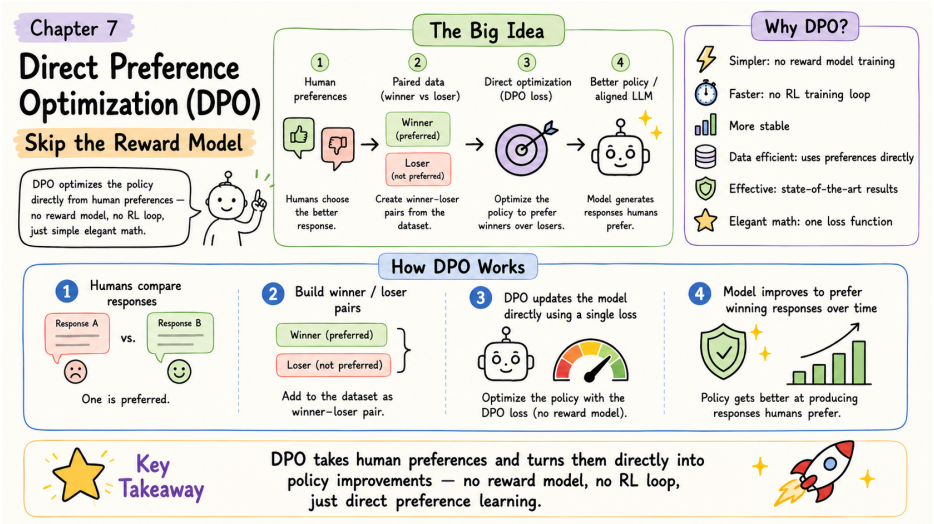

Figure 7.1: Direct Preference Optimization at a glance – skip the reward model and optimize the policy directly from preference pairs.

The Reparameterization Trick

The derivation starts from the same place as Chapter 6: the Bradley-Terry model of preferences and the KL-constrained reward maximization objective. Recall the RLHF objective from Section 6.3.1:

$$\max_{\pi_\theta} \mathbb{E}_{x,y}\left[r_\phi(x,y)\right] - \beta \cdot \mathbb{E}_x[\mathrm{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})]$$

This objective has a known closed-form solution. If you could solve it perfectly, the optimal policy π^* would take a specific mathematical form. DPO's insight is to write down that form, rearrange it, and substitute back into the Bradley-Terry preference model from Section 6.2.2. When you do, something remarkable happens: the reward model cancels out entirely.

The DPO Reparameterization in Three Steps

Step 1: Write down the optimal policy

$$\pi^*(y|x) = \frac{1}{Z(x)}\,\pi_{\text{ref}}(y|x)\,\exp\left(\frac{r(x,y)}{\beta}\right)$$

The best possible policy takes the reference model and reweights each response by the exponential of its reward. $Z(x)$ is a normalizing constant that ensures probabilities sum to 1.

Step 2: Solve for the reward

$$r(x,y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

Rearranging Step 1 gives us the reward as a function of the policy ratio. The reward for any response is proportional to how much more likely the optimal policy makes it compared to the reference.

Step 3: Substitute into Bradley-Terry

$$P(y_w \succ y_l) = \sigma\left(\beta \log \frac{\pi(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

When we plug the reward expression into the preference model from Chapter 6, the partition function $Z(x)$ appears in both terms and cancels. We are left with preference probability expressed entirely in terms of policy ratios. No reward model anywhere.

Why does $Z(x)$ cancel? In the Bradley-Terry model, the preference probability depends on the *difference* between the rewards for the winner and loser: $r(x, y_w) - r(x, y_l)$. When

we substitute the Step 2 expression for both rewards, each contains the same $\beta \log Z(x)$ term. Subtracting one from the other eliminates $Z(x)$ completely. This is the mathematical core of DPO: a term that would be intractable to compute simply drops out of the equation.

What this means in practice. Recall from Chapter 6 that RLHF requires four models: the policy, the reference, the reward model, and the value function. The reparameterization trick shows that we can express the preference probability using only the policy and the reference. The reward model and value function are no longer needed.

Component	RLHF (Chapter 6)	DPO
Policy π_θ	Yes (trainable)	Yes (trainable)
Reference π_{ref}	Yes (frozen)	Yes (frozen)
Reward model r_ϕ	Yes (frozen)	**Not needed**
Value function V	Yes (trainable)	**Not needed**
Total models	4	2

The key insight. We do not need to explicitly model the reward function $r(x, y)$. The policy ratio $\frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ already encodes all the information about what makes responses good.

If π_θ makes a response more likely than π_{ref} does, that response has high *implicit* reward. If π_θ makes it less likely, the response has low implicit reward. Training can adjust π_θ directly on preference pairs, without ever computing an explicit reward score.

The DPO Loss Function

The reparameterization gives us a way to express preference probability in terms of policy ratios. To turn this into a training objective, we apply maximum likelihood: find the policy parameters θ that make the observed preference data most likely.

Training the Policy Directly on Preferences

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

In plain English: Adjust the policy so that for every preference pair, the winner's implicit reward is higher than the loser's.

Breaking it apart:

Component	Purpose
$\beta \log \frac{\pi_\theta(y_w\|x)}{\pi_{\text{ref}}(y_w\|x)}$	Implicit reward for the winner. How much has the policy increased the winner's likelihood relative to the reference?
$\beta \log \frac{\pi_\theta(y_l\|x)}{\pi_{\text{ref}}(y_l\|x)}$	Implicit reward for the loser. How much has the policy increased the loser's likelihood relative to the reference?
$\sigma(\text{winner} - \text{loser})$	Probability that the policy assigns higher implicit reward to the winner than the loser. Should be close to 1 if training is working.
$\log(\cdot)$	Log likelihood. Heavily penalizes confident wrong predictions (when the model thinks the loser is better).
$-$	Flip sign so that minimizing the loss maximizes correct preference predictions.
$\mathbb{E}[\cdot]$	Average over all preference pairs in the dataset.

Compare with the reward model loss from Chapter 6:

Reward model loss (Chapter 6):

$$-\mathbb{E}\left[\log \sigma\left(r_\phi(x, y_w) - r_\phi(x, y_l)\right)\right]$$

DPO loss (this chapter):

$$-\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

The structure is identical. The only difference is what plays the role of the "score": explicit reward model outputs r_ϕ in RLHF, versus implicit rewards from policy ratios $\beta \log \frac{\pi_\theta}{\pi_{\text{ref}}}$ in DPO. Same framework, different parameterization.

Step-by-Step Walkthrough

Let us trace through DPO training on a single preference pair, computing every number.

Preference pair:

- **Prompt (x):** "Explain photosynthesis"
- **Winner (y_w):** "Plants use sunlight to make food from CO_2 and water" (10 tokens)
- **Loser (y_l):** "Photosynthesis is the biochemical process involving chloroplast thylakoids…" (12 tokens)

The winner is simpler and clearer. We want DPO to learn this preference.

Step 1: Compute token probabilities for both responses

For each response, both the current policy π_θ and the frozen reference π_{ref} assign a probability to every token. For simplicity, we use average per-token probabilities.

Winner (10 tokens):

Model	Avg per-token probability	Full sequence probability
π_θ (current)	0.22	0.22^{10}
π_{ref} (reference)	0.20	0.20^{10}

Loser (12 tokens):

Model	Avg per-token probability	Full sequence probability
π_θ (current)	0.17	0.17^{12}
π_{ref} (reference)	0.19	0.19^{12}

Notice what is already happening: the current policy assigns slightly higher probability to each winner token (0.22 vs 0.20) and slightly lower probability to each loser token (0.17 vs 0.19) compared to the reference. The policy is starting to differentiate, but only slightly.

Step 2: Compute log ratios (the implicit rewards)

Winner log ratio:

$$\log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} = 10 \times \log \frac{0.22}{0.20} = 10 \times 0.095 = 0.95$$

The current policy makes the winner 0.95 log-units more likely than the reference does. This is a positive implicit reward: the policy "likes" this response.

Loser log ratio:

$$\log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)} = 12 \times \log \frac{0.17}{0.19} = 12 \times (-0.111) = -1.33$$

The current policy makes the loser 1.33 log-units *less* likely than the reference. Negative implicit reward: the policy has already started moving away from this response.

Step 3: Compute the preference logit

With $\beta = 0.1$:

$$\begin{aligned}\text{logit} &= \beta \times (\text{winner log ratio} - \text{loser log ratio}) \\ &= 0.1 \times (0.95 - (-1.33)) \\ &= 0.1 \times 2.28 = 0.228\end{aligned}$$

Step 4: Convert to preference probability

$$P(\text{winner} \succ \text{loser}) = \sigma(0.228) = \frac{1}{1 + e^{-0.228}} = 0.557 \quad \text{or } 55.7\%$$

The model is only slightly better than chance at distinguishing the winner from the loser. Not great yet.

Step 5: Compute the loss

$$\mathcal{L}_{\text{DPO}} = -\log(0.557) = 0.585$$

Recall from Chapter 6 that a random model produces a loss of $-\log(0.5) = 0.693$. Our loss of 0.585 is only slightly better than random, confirming that the model has barely begun to learn this preference.

Step 6: Gradient descent updates π_θ

The gradients push in two directions simultaneously. They **increase** $\pi_\theta(y_w|x)$, making the winner tokens more likely under the policy. And they **decrease** $\pi_\theta(y_l|x)$, making the loser tokens less likely. This is the core learning dynamic of DPO: every preference pair teaches the model to widen the gap between winner-style and loser-style responses.

After training on 50,000 preference pairs

Running the same computation on this preference pair after training:

Quantity	Before training	After training
Winner log ratio	+0.95	+2.8
Loser log ratio	−1.33	−2.5
Difference	2.28	5.3
Logit ($\beta = 0.1$)	0.228	0.53
$P(\text{winner} \succ \text{loser})$	55.7%	62.9%
Loss	0.585	0.464

The model has learned to prefer simpler explanations, assign higher probability to clear and accessible language, and assign lower probability to overly technical responses. All of this happened without an explicit reward model. The policy learned directly from preference pairs.

Why DPO works. Every preference pair teaches the model: "This style of response is better than that style." The policy ratios $\frac{\pi_\theta}{\pi_{\text{ref}}}$ encode implicit rewards that are mathematically equivalent to the explicit rewards that a separate reward model would produce. Training automatically makes winner log ratios larger and loser log ratios smaller.

The training loop is remarkably simple: load a batch of preference pairs, compute log probabilities under π_θ and π_{ref}, compute the DPO loss, backpropagate. No generation step, no reward scoring, no advantage estimation, no PPO clipping. It looks and feels like supervised learning, but it optimizes the same objective as RLHF.

DPO vs RLHF: A Head-to-Head Comparison

Now that you understand both methods, let us compare them directly. The right choice depends on your specific constraints.

Dimension	RLHF with PPO (Chapter 6)	DPO (this chapter)
Models in memory	4 (policy, reference, reward, value)	2 (policy, reference)
Training pipeline	Three separate stages (SFT → reward model → PPO)	Two stages (SFT → DPO)
Training stability	Sensitive to many hyperparameters (clip range, PPO epochs, GAE λ, etc.)	Relatively stable; fewer hyperparameters to tune
Data requirements	Can generate new data on-the-fly (online)	Requires a fixed dataset of preference pairs (offline)
Exploration	Policy generates new responses each iteration; can discover novel strategies	No generation during training; limited to strategies present in the preference data
Reward hacking	Risk of exploiting reward model weaknesses; KL penalty helps but is imperfect	Different risk profile: can overfit to preference data artifacts instead
Compute cost	High (generation + scoring + RL updates per batch)	Lower (forward passes through two models, standard backprop)
Implementation	Complex (PPO loop, advantage estimation, multiple model synchronization)	Simple (looks like supervised fine-tuning with a modified loss)
Peak performance	Slightly higher ceiling with abundant compute and careful tuning	Comparable in most settings; may lag behind in high-compute regimes

When does RLHF still win?

RLHF with PPO retains advantages in two scenarios. First, when you have abundant compute and engineering resources, PPO's online exploration can discover response strategies that are not present in any static preference dataset. The policy generates novel responses, the reward model scores them, and training reinforces whatever works. DPO cannot do this because it never generates responses during training.

Second, when the task distribution shifts between preference data collection and deploy-

ment, PPO adapts better because it continuously generates and evaluates responses from the current policy. DPO trains on a fixed dataset, so if that dataset does not cover the prompts the model will face in production, performance can degrade.

When is DPO the better choice?

For most practitioners, DPO is the pragmatic default. If you are working with limited GPU memory (a single consumer GPU or a free Colab T4), DPO's two-model footprint may be the only viable option. If you want a simpler, more reproducible training pipeline, DPO eliminates the complex PPO loop and its many hyperparameters. If you already have a good preference dataset (or can generate one synthetically using a stronger model), DPO can match RLHF performance with less effort.

The practitioner's rule of thumb. Start with DPO. It is simpler to implement, faster to iterate, and easier to debug. If you observe specific problems that DPO cannot solve (distribution mismatch, insufficient exploration, performance plateau), then consider the more complex alternatives: Online DPO (Chapter 8) adds exploration back, and PPO (Chapter 6) provides the full online RL loop.

The β Parameter and Practical Considerations

Understanding β in DPO

In Chapter 6, β appeared as the coefficient on the KL penalty in the RLHF objective. In DPO, β appears in the loss function directly, but it plays an analogous role: it controls how far the policy is allowed to deviate from the reference model.

Formally, β scales the log ratios in the DPO loss:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)\right]$$

A larger β amplifies the log ratios, making the loss more sensitive to small differences

between the policy and reference. A smaller β dampens the log ratios, allowing the policy to deviate further before the loss reacts.

Effect of β on the same preference pair:

Using the log ratios from our photosynthesis example (winner: +0.95, loser: −1.33, difference: 2.28):

β	**Logit**	$P(\text{w} \succ \text{l})$	**Loss**	**Interpretation**
0.01	0.023	50.6%	0.690	Almost no signal. Policy barely distinguishes winner from loser. Loss
0.05	0.114	52.8%	0.637	Weak signal. Slow learning.
0.10	0.228	55.7%	0.585	Moderate signal. Standard starting point.
0.20	0.456	61.2%	0.490	Strong signal. Faster learning but more sensitivity.
0.50	1.140	75.8%	0.277	Very strong signal. Risk of overfitting to noisy preferences.

Tuning β in DPO

Too small ($\beta < 0.05$): The loss function is nearly flat. Gradients are tiny, learning is extremely slow, and the model may fail to converge. Equivalent to a very tight leash–the policy barely moves from the reference.

Just right ($\beta = 0.1$ to 0.2): The standard range for most tasks. The loss provides clear gradients without being oversensitive to noise in the preference data. Start here.

Too large ($\beta > 0.5$): The loss reacts strongly to every preference pair, including noisy or mislabeled ones. The policy can overfit to artifacts in the data (such as preferring longer responses because annotators tended to pick longer answers). Equivalent to a loose leash–the policy drifts far from the reference.

Relationship to Chapter 6: In RLHF, β controls the KL penalty strength. A large β in RLHF means a tight leash (conservative policy). In DPO, the rela-

tionship is *inverted*: a large β amplifies the implicit reward signal, effectively giving the policy more room to move. This difference can be confusing when switching between the two frameworks, so be careful when reading papers that use both.

Other Hyperparameters

Beyond β, several practical choices affect DPO training quality.

Learning rate. DPO is more sensitive to learning rate than standard supervised fine-tuning. A rate that works well for SFT (e.g., 2×10^{-5}) is often too high for DPO. Typical starting points are 5×10^{-7} to 5×10^{-6}. If the loss spikes or the model degenerates, reduce the learning rate first.

Batch size. Because DPO averages over preference pairs, larger batches give more stable gradient estimates. A batch size of 32 to 64 preference pairs is common. If you are memory-constrained, use gradient accumulation to simulate larger batches.

Reference model handling. The reference model π_{ref} must remain fixed throughout training. In practice, this means loading a second copy of the SFT model in inference mode. With LoRA, you can share the base model weights and only keep separate adapter weights for π_θ, which significantly reduces memory. Some implementations precompute π_{ref}'s log probabilities for all responses in the preference dataset before training begins, eliminating the need to keep π_{ref} in memory during the DPO loop.

Number of epochs. DPO tends to overfit if trained for too many epochs on the same preference data. One to three epochs is typical. Monitor the validation loss and stop when it begins to increase.

Common failure modes and fixes:

Loss collapses to near zero: The model has memorized the preference pairs rather than learning general patterns. Reduce epochs, increase β, or

add more diverse preference data.

Loss oscillates wildly: Learning rate is too high or batch size is too small. Reduce learning rate by 2 to 5× and increase gradient accumulation steps.

Model becomes repetitive: The policy has drifted too far from the reference. Increase β to strengthen the implicit KL regularization, or reduce the learning rate.

Model always generates longer responses: The preference data may have a length bias (annotators often prefer longer answers). Consider length-normalized rewards or filtering the preference data to remove length-correlated pairs.

Limitations and What Comes Next

DPO is simpler, cheaper, and easier to implement than RLHF. But it is not a free lunch. Understanding its limitations is essential for knowing when to reach for something more powerful.

Offline only: no exploration. DPO trains on a fixed dataset of preference pairs. The policy never generates new responses during training, which means it cannot discover strategies that are not already represented in the data. If the preference dataset was collected from a weaker model, DPO can only learn to rank within that model's capability range, not exceed it. This is the fundamental limitation of offline preference learning: you are bounded by the quality and diversity of your data.

Sensitive to data quality. Because DPO has no reward model to provide a smooth scoring function, it relies entirely on the raw preference labels. Noisy labels (where annotators disagreed or made mistakes) directly inject noise into the training signal. RLHF is somewhat more robust because the reward model smooths over individual labeling errors by learning a continuous scoring function from the aggregate data.

Implicit reward can be miscalibrated. DPO's "implicit reward" (the log policy ratio) is not a calibrated score in the way that a trained reward model's output is. This means you cannot easily use DPO's implicit reward for tasks like best-of-N sampling or rejection sampling, where you need to compare scores across different prompts. The implicit reward is meaningful only as a relative signal within a single preference pair.

What comes next. Each of these limitations has motivated extensions that we will cover in later chapters:

Online DPO and iterative alignment (Chapter 8) address the offline limitation by alternating between generating new responses with the current policy and training on the resulting preference pairs. This recovers some of the exploration benefits of RLHF while keeping DPO's simplicity.

Modern algorithm variants (Chapter 10) address specific weaknesses of vanilla DPO. IPO adds explicit regularization to prevent overfitting. KTO works with binary feedback (thumbs up or down) instead of paired comparisons, which is cheaper to collect and more robust to noise. ORPO combines SFT and preference learning into a single training stage. Each variant makes a different trade-off, and Chapter 10 provides a decision framework for choosing among them.

The DPO chapter in context.

You now have three methods in your toolkit:

SFT (Chapter 5): Teaches basic competence by imitating demonstrations. Simple but limited to the quality of training data.

RLHF with PPO (Chapter 6): Surpasses demonstrations using reward model guidance. Powerful but expensive and complex (four models, many hyperparameters).

DPO (this chapter): Achieves much of RLHF's benefit at a fraction of the cost and complexity. Two models, supervised-style training, comparable

results.

The next chapter introduces Online DPO, which addresses DPO's biggest weakness (the offline limitation) while preserving its simplicity. Then Chapter 9 covers reward modeling in greater depth, and Chapter 10 surveys the full algorithm landscape.

The companion code for this chapter is at `github.com/arunpshankar/packt-final` – swap `github.com` for `githubtocolab.com` to open it directly in Colab.